



Student Course Management System

C++ Console Application | Object-Oriented Programming

OOP

STL Containers

Inheritance

Algorithms

PROJECT OVERVIEW

OBJECTIVE

Develop a C++ console application to manage students and their enrolled courses.

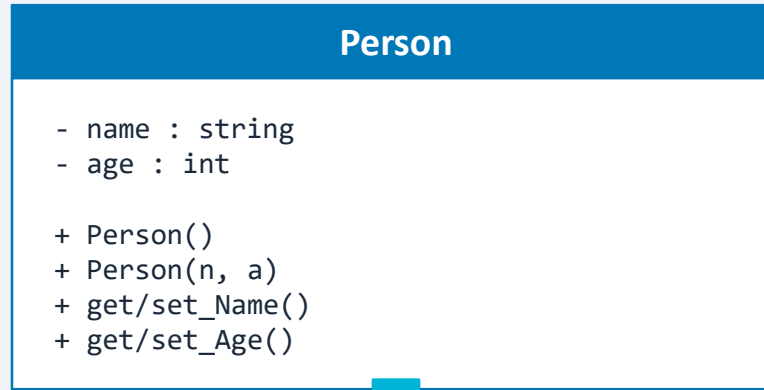
Core Goals:

- Apply C++ Basics & OOP principles
- Use STL containers effectively
- Implement search and sort algorithms
- Write modular, organized code

KEY FEATURES

- Menu-driven console interface
- Add / Remove / Search students
- Display all students
- Enroll in courses
- Display student courses
- Sort students by GPA
- Unique ID validation
- GPA range enforcement (0–4)

CLASS ARCHITECTURE (UML)



extends

Student (inherits Person)

- ID : int
- GPA : float
- Course : set<string>
- + students : vector<Student>

- + addstudent()
- + removestudent()
- + searchstudent()
- + enrollstudent()
- + sort_Students()

STL Containers used:

vector

set

CORE FUNCTIONS



Add Student

Validates unique ID, collects name & GPA (0–4), stores in vector



Remove Student

Finds student by ID using loop, erases from vector



Search Student

Uses `find_if()` algorithm to locate student by ID



Display All

Iterates vector and prints ID, Name, GPA for all students



Enroll in Course

Inserts course into `set<string>` — auto-prevents duplicates



Sort by GPA

Uses `sort()` with a lambda comparator (descending order)

OOP CONCEPTS APPLIED

Encapsulation

Private data members (name, age, ID, GPA, Course) protected by public getter/setter methods. No direct access from outside the class.

Inheritance

Student inherits from Person. Reuses name & age attributes and getter/setter methods without code duplication.

Constructors

Person provides default and parameterized constructors. Ensures proper initialization of objects when created.

Polymorphism

Display() method in Student extends inherited get_Name() from Person, combining parent and child class data.

STL CONTAINERS & ALGORITHMS

vector<Student>

Stores all students. Supports dynamic add/remove and iteration.

```
students.push_back()  
students.erase()  
students.begin()/end()
```

set<string>

Stores enrolled courses per student. Automatically prevents duplicate course names.

```
Course.insert(name)  
for (auto c : Course)
```

STL ALGORITHMS

find_if()

Searches vector for student by ID using a lambda predicate.

sort()

Sorts students by GPA in descending order using a lambda comparator.

CODE HIGHLIGHTS

addstudent() — Validation

```
void Student::addstudent() {
    cin >> id;
    for (auto s : students) {
        if (s.get_ID() == id) {
            cout << "Already Exists";
        }
    }
    students.push_back(stu);
}
```

searchstudent() — find_if()

```
void Student::searchstudent() {
    auto stu = find_if(
        students.begin(),
        students.end(),
        [&](Student& s) {
            return s.get_ID()==id;
        });
    stu->Display();
}
```

sort_Students() — Lambda Sort

```
void Student::sort_Students() {
    sort(students.begin(),
        students.end(),
        [](Student& a, Student& b){
            return a.get_GPA() >
                b.get_GPA();
        });
}
```

add_Course() — set Insert

```
// Courses stored in set<string>
// Duplicates auto-prevented!

void Student::add_Course(
    string course) {
    Course.insert(course);
}
```

PROGRAM FLOW & MENU

Console Output

```
Student Course Management System  
=====
```

1. Add Student
2. Remove Student
3. Search Student
4. Display All Students
5. Enroll Student in Course
6. Show Student Courses
7. Sort Students by GPA
8. Exit

```
Enter choice: _
```

1

Program Start

main() creates Student object, displays menu in loop

2

User Input

switch statement routes to correct member function

3

Validation

ID uniqueness & GPA range checked before saving

4

Data Storage

Student added to vector<Student> or course to set<string>

5

Output

Results displayed to console; loop continues until Exit

PROJECT SUMMARY



OOP Design

Classes, Encapsulation, Inheritance, Constructors



STL Containers

`vector<Student>` + `set<string>` — 2 containers used



STL Algorithms

`find_if()` and `sort()` with lambda expressions



Input Validation

Unique ID check, GPA range 0.0–4.0 enforced



Duplicate Prevention

`set` automatically prevents duplicate courses



All Menu Functions

Add, Remove, Search, Display, Enroll, Sort